Gemini Fast

This is a significant hardware design project! You are essentially building the "brain" and "skeleton" of a **Simple Instruction Set Computer (SISC)**.

[cite_start]To get this right, we have to bridge the gap between the high-level schematic you see in the image [cite: 1] and the low-level Verilog code.

📋 The Mini TODO List

Here is the high-level flight plan for Part 1:

1. [cite_start]**Define Internal Wiring (`sisc.v`):** Create the "nerve system" to connect all modules based on the diagram[cite: 1, 53].
2. [cite_start]**Instantiate Components (`sisc.v`):** Plug in the ALU, Register File, Mux, Status Register, and Control Unit[cite: 53].
3. [cite_start]**Implement Control Logic (`ctrl.v`):** Program the "brain" to set `alu_op`, `rf_we`, and `wb_sel` for each instruction[cite: 52].
4. [cite_start]**Add Monitoring (`sisc.v`):** Set up the `$monitor` task to track registers and signals for grading[cite: 67, 68].
5. [cite_start]**Verify & Debug:** Run the provided testbench (`sisc_tb_p1.v`) and check the output against expected values[cite: 54, 77].

🛠 Step 1: Define Internal Wiring in `sisc.v`

[cite_start]In your `sisc.v` file, you currently have a module with inputs but no internals[cite: 218]. Think of this step as laying down the copper traces on a circuit board. You need to declare `wire` signals for every line you see in the diagram that isn't a top-level input.

How to do it:

[cite_start]Look at the diagram [cite: 1] and the provided module headers to name your wires. You will need:

- **Instruction Breaks:** The 32-bit `ir` input needs to be split.
  - [cite_start] `ir[31:28]` → Opcode (goes to CTRL)[cite: 13, 161].
  - [cite_start] `ir[27:24]` → ALU "mm" or Function bits (goes to CTRL)[cite: 13, 161].
  - [cite_start] `ir[23:20]` → Write Register address (goes to RF)[cite: 26, 198].

- [cite_start] `ir[19:16]` → Read Reg A address (goes to RF)[cite: 20, 198].
- [cite_start] `ir[15:12]` → Read Reg B address (goes to RF)[cite: 34, 198].
- [cite_start] `ir[15:0]` → Immediate value (goes to ALU)[cite: 40, 80].

- **Control Signals (The "Orange" Lines):**
  - [cite_start] `wire [3:0] alu_op_wire;` (From CTRL to ALU)[cite: 9, 80].
  - [cite_start] `wire rf_we_wire;` (From CTRL to RF)[cite: 16, 198].
  - [cite_start] `wire wb_sel_wire;` (From CTRL to MUX32)[cite: 10, 189].
- **Data Paths:**
  - [cite_start] `wire [31:0] rsa_wire, rsb_wire;` (From RF to ALU)[cite: 21, 27, 80].
  - [cite_start] `wire [31:0] alu_result_wire;` (From ALU to MUX32)[cite: 38, 189].
  - [cite_start] `wire [31:0] write_data_wire;` (From MUX32 back to RF)[cite: 35, 198].
  - [cite_start] `wire [3:0] stat_wire, stat_en_wire, stat_out_wire;` (Between ALU, STATREG, and CTRL)[cite: 14, 19, 80, 246].

Would you like me to move on to Step 2 (Instantiating the modules), or do you have questions about how these wires correspond to the diagram?

Gemini Fast

Step 2: Instantiate the Components in `sisc.v`

Now that the "wires" are ready, you need to plug in the physical modules. Think of this like placing components on a motherboard and soldering those wires to the correct pins.

You will use the **Module Name** followed by an **Instance Name** (a name you make up, like `my_alu`), and then map the ports. [cite_start]Referencing the provided files [cite: 80, 161, 189, 198, 246] [cite_start]and the diagram[cite: 1], here is how you connect them:

1. The Register File (`rf`)

This is the storage unit.

- **Connections:**
  - [cite_start] `clk` connects to the system `clk` [cite: 206].
  - [cite_start] `read_rega` connects to `ir[19:16]` [cite: 199, 207].
  - [cite_start] `read_regb` connects to `ir[15:12]` [cite: 200, 207].
  - [cite_start] `write_reg` connects to `ir[23:20]` [cite: 201, 207].

- [cite_start] `write_data` connects to the output of your MUX[cite: 201, 207].
- [cite_start] `rf_we` connects to the `rf_we` control signal from the CTRL unit[cite: 201, 208].
- [cite_start] `rsa` and `rsb` connect to the 32-bit wires leading to the ALU[cite: 203, 208].

2. The Arithmetic Logic Unit (`alu`)

This performs the math.

- **Connections:**
  - [cite_start] `rsa` and `rsb` come from the RF[cite: 82, 83, 108].
  - [cite_start] `imm` connects to the immediate value `ir[15:0]` [cite: 84, 109].
  - [cite_start] `alu_op` comes from the CTRL unit[cite: 86, 109].
  - [cite_start] `funct` comes from `ir[27:24]` (the "mm" or function bits)[cite: 94, 109].
  - [cite_start] `c_in` comes from the Status Register (specifically the Carry bit, `stat_out[3]`) [cite: 97, 108].
  - [cite_start] `alu_result` connects to the wire going to the MUX[cite: 98, 109].
  - [cite_start] `stat` and `stat_en` go to the Status Register[cite: 100, 105, 110].

3. The Control Unit (`ctrl`)

This is the "brain" that decides what happens in each state.

- **Connections:**
  - [cite_start] `opcode` connects to `ir[31:28]` [cite: 164].
  - [cite_start] `mm` connects to `ir[27:24]` [cite: 164].
  - [cite_start] `stat` connects to the output of the Status Register[cite: 164].
  - [cite_start]Outputs `rf_we` , `alu_op` , and `wb_sel` connect to their respective components[cite: 164].

4. The 32-bit Multiplexer (`mux32`)

This decides what gets written back to the registers.

- **Connections:**
  - [cite_start] `in_a` (sel=0) connects to the `alu_result` [cite: 190, 196].
  - [cite_start] `in_b` (sel=1) connects to the value `0` (for now, based on the diagram)[cite: 191, 197].
  - [cite_start] `sel` connects to the `wb_sel` signal from CTRL[cite: 192, 195].
  - [cite_start] `out` goes back to the `write_data` port of the RF[cite: 193].

5. The Status Register (`statreg`)

This remembers the results of the last math operation (like if it was zero or negative).

- **Connections:**
  - [cite_start] `in` connects to the `stat` output of the ALU[cite: 247, 252].
  - [cite_start] `enable` connects to the `stat_en` output of the ALU[cite: 250, 252].
  - [cite_start] `out` provides the status bits to the CTRL unit and ALU[cite: 251, 253].

Does this connection logic make sense, or would you like to review the specific Verilog syntax for one of these instantiations before we move to Step 3 (The Control FSM)?

Gemini Fast

Step 3: Implement the Control FSM (`ctrl.v`)

This is the most critical part of the project. [cite_start]The Control Unit acts as the brain, orchestrating how data moves through the processor based on the current **state** and the **instruction** it sees[cite: 52].

[cite_start]Since the `ctrl.v` file already has the state transition logic (moving from `fetch` to `decode` to `execute`, etc.) [cite: 178, 179, 180, 181, 182][cite_start], your job is to define what the **orange control signals** (`alu_op` , `rf_we` , and `wb_sel`) should be during those states[cite: 52].

1. Understanding the Signal "Jobs"

- **`alu_op` (4 bits):** This tells the ALU what math to do. [cite_start]Bit 0 enables status updates, while Bits 3:1 select the operation (e.g., `000` for register-to-register, `001` for register-to-immediate)[cite: 86, 87, 88, 89].
- **`rf_we` (1 bit):** Register File Write Enable. [cite_start]If this is `1` , the value at the MUX output is saved into the destination register[cite: 201, 202].
- **`wb_sel` (1 bit):** Write-Back Select. [cite_start]This tells the `MUX32` whether to send the `alu_result` (0) or a constant `0` (1) back to the register file[cite: 190, 191, 192].

---

2. How to Map the Instructions

[cite_start]You need to handle instructions like `ADD` , `ADI` (Add Immediate), and `SUB` [cite: 65]. [cite_start]Here is the logic you will implement in the `always @(present_state, opcode)` block[cite: 184]:

Instruction	[cite_start]Opcode [cite: 166, 167]	[cite_start] `alu_op` (3:1) Logic [cite: 88, 89, 90]	`wb_sel`	`rf_we`
ADD / SUB / etc.	`REG_OP` (1)	`3'b000` (Use `funct` bits)	`0`	`1` (at Writeback)
ADI	REG_IM (2)	3'b001 (Use Immediate)	0	1 (at Writeback)
NOP	`NOOP` (0)	`3'b110` (Pass RSA)	`0`	`0` (No write)

---

3. State-Specific Timing

Control signals shouldn't be "on" all the time. For Part 1, follow this pattern:

1. **FETCH / DECODE:** Keep `rf_we` at `0`. The CPU is just figuring out what to do.
2. **EXECUTE:** Set the `alu_op` so the ALU can calculate the result.
3. **WRITEBACK:** This is when you set `rf_we = 1`. [cite_start]If you set it too early, you might save the wrong data before the ALU is finished[cite: 201, 216].

> [cite_start]**Important Note:** For `alu_op`, remember that bit 0 is the "Status Update" bit[cite: 87]. [cite_start]If you want the `statreg` to save the results of a calculation (like a carry or zero flag), `alu_op[0]` must be `1` [cite: 105, 160].

Do you understand how to assign these signals based on the state and opcode, or should we look at a specific example like the `ADI` instruction?

Gemini Fast

Step 4: Add Monitoring to `sisc.v`

In Verilog, the `$monitor` statement is a powerful tool for "watching" signals in real-time. [cite_start]Since your processor doesn't have a screen or a printer, this is how you prove to your instructor (and yourself) that the registers are actually changing[cite: 67, 76].

1. What to Monitor

[cite_start]You are required to track specific signals every time one of them changes[cite: 68]:

- **Instruction Register:** `ir`
- **Control Signals:** `alu_op`, `wb_sel`, and `rf_we`.
- **Data Path:** The 32-bit line connected to the `write_data` input of the register file.

- [cite_start]**Internal Registers:** R1, R2, R3, R4, and R5[cite: 68].

2. Accessing Internal Signals (The Dot Operator)

Registers like `R1` are buried inside the `rf` (Register File) module. [cite_start]To see them from the top-level `sisc.v` file, you must use **hierarchical naming**[cite: 69].

[cite_start]If you named your register file instance `my_rf` in Step 2, you would access the first register using `my_rf.ram_array[1]` [cite: 70].

3. Implementation Logic

[cite_start]Place the `$monitor` statement inside an `initial` block within your `sisc.v` file[cite: 219]. It should look something like this:

```verilog
initial begin
  $monitor("Time=%d IR=%h R1=%h R2=%h ALU_OP=%b",
           $time, ir, my_rf.ram_array[1], my_rf.ram_array[2], alu_op_wire);
end
```

[cite_start]*(Note: You'll need to expand this to include all 10 required signals listed in the project requirements[cite: 68].)*

---

4. Why this is the "Final Piece"

Once this is added, running the testbench (`sisc_tb_p1.v`) will automatically print a text log to your console. [cite_start]You can then compare that log to the "expected results" provided in the testbench comments—for example, checking if `R1` becomes `00000001` after the `ADI` instruction[cite: 239].

Does the concept of hierarchical naming (using the dot operator) make sense, or would you like to move on to the final Step 5: Verification & Debugging?

Gemini Fast

Step 5: Verification & Debugging

[cite_start]The final stage is ensuring your machine actually works by running the provided testbench, `sisc_tb_p1.v` , and analyzing the output generated by your `$monitor` statement[cite: 54, 67, 76].

1. The Clock and Reset Cycle

- [cite_start]The testbench initializes `rst_f` to 0 for 20 ns (two cycles) to clear the state[cite: 62, 227, 228].
- [cite_start]The first instruction is delayed by 85 ns to align with the first cycle after reset[cite: 63, 229].
- [cite_start]Each subsequent instruction is spaced by 50 ns to allow the 5-cycle FSM (`fetch` through `writeback`) to complete[cite: 63, 64, 182].

2. Checking the "Golden" Results

[cite_start]The testbench contains comments showing what the registers should look like after the first block of instructions[cite: 239]:

- [cite_start]**R1:** `00000001` (From the `ADI` instruction)[cite: 229, 239].
- [cite_start]**R2:** `FF000008` [cite: 239].
- [cite_start]**R3:** `FE000000` [cite: 239].
- [cite_start]**R4:** `FE000011` [cite: 239].
- [cite_start]**R5:** `FF000019` [cite: 239].

3. Status Bit Verification

[cite_start]You must also verify that the `statreg` correctly captures flags during arithmetic[cite: 240]. For example:

- [cite_start] `SUB R2, R1, R1` should result in a **Zero** flag (`STAT: 0001`)[cite: 241].
- [cite_start] `SUB R2, R0, R1` should result in **Negative** and **Borrow** flags (`STAT: 1010`)[cite: 242].

4. Common Troubleshooting Tips

- [cite_start]**The "Halt" Problem:** Ensure your `ctrl.v` triggers the `$stop` command when the `HLT` (opcode `15`) instruction is reached, or your simulation will run forever[cite: 65, 66, 186, 187].
- **Signal X or Z:** If your monitor shows `xxxxxxxx` , a wire in `sisc.v` is likely disconnected. If it shows `zzzzzzzz` , you might have a tri-state issue or a naming typo.
- [cite_start]**Timing Offsets:** If the registers update one cycle too late, check if you are asserting `rf_we` during the `writeback` state or accidentally during `execute` [cite: 181, 182].

You have now seen the full roadmap for Part 1! Would you like me to provide a template for the `sisc.v` structural code to help you get started with the wiring?